

Hash Table and Heap

Author: Eric Zhou

Email: eric_ds_zhou@outlook.com

Date: Oct 5, 2026

Introduction

This technical note summarizes the tools and logic used in Hash Table and heap.

Table of contents

- What Is a Good Hash Table?
- Type of Hash Tables
 - Chained Hashing
 - Open Address Hashing
- Time complexity about Hash Table
- Table Doubling
- What Is a valid heap?
- Heap operation
 - Insert
 - Replace
- Analysizing Alogrithms Using Heap
- Bonus: Leetcode Two Sum

What Is a Good Hash Table

According to the note, Good hash functions should have the following properties:

(1) All hash table locations are equally likely to be accessed.

We should try our best to distribute keys uniformly across all buckets.

Bad Hash Table:

$$h(K) = (9K) \bmod 78$$

Since $\gcd(9, 78) = 3$, the hash function can only produce multiples of 3 (0, 3, 6, ..., 75). Therefore, only 1/3 of the buckets can be used.

(2) Keys with a "regular" pattern should not be mapped to the same locations.

Bad Hash Table:

$$h(K) = K \bmod 10$$

We can easily see that keys 10, 20, and 30 will all be mapped to bucket 0, resulting in multiple collisions.

In addition, we also need to remember that the the size of hash table should be bigger than the number to be mod, or some indices will be invalid.

- Type of Hash Tables

Chained Hashing

Open Address Hashing

Time complexity about Hash Table

Table Doubling

What Is a valid heap?

Heap operation

Insert

Replace

Analysizing Alogrithms Using Heap

Bonus: Leetcode Two Sum

Since we learned Two Sum with hash tables, why don't we take a look at the actual LeetCode Two Sum problem?

Question:

You are given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Intuition

It's clear that for each element `A[i]`, we need to find its pair, `target - A[i]`.

The easiest way to solve this is to use two `for` loops to check all possible pairs in the array. However, the time complexity of this approach is $\Theta(n^2)$.

Therefore, we can use a hash table to improve the efficiency.

Approach

Since a well-implemented hash table supports dictionary operations such as `member`, `retrieve`, `add`, and `delete` in expected $\Theta(1)$ time, we can use a hash table to store the numbers we have already seen.

In Java, we use `containsKey()`, `get()`, and `put()` to perform these dictionary operations.

For each `nums[i]`, we first calculate:

$$need = target - nums[i]$$

Then, we check whether `need` is already in the hash table.

- If it is, we use `get()` to retrieve its index and return the two indices.
- Otherwise, we use `put()` to store `nums[i]` as the key and `i` as its value.

Time complexity: $\Theta(n)$

Code

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];

            if (map.containsKey(need)) {
                return new int[]{map.get(need), i};
            }

            map.put(nums[i], i);
        }

        return new int[]{};
    }
}
```

Reference List

1. "CSE2331_Midterm_3_Review" Created by Professor Painter.
2. "CSE2331_Heaps_Homework" Created by Professor Painter.
3. "CSE2331_Hashing_Homework" Created by Professor Painter.
4. LeetCode. "1. Two Sum." <https://leetcode.com/problems/two-sum/>